

# Statistics 651

## MCMC

Winter 2026

# Objectives

1. Construct Metropolis-Hastings samplers
2. Construct hybrid samplers with M-H steps within a Gibbs sampler
3. Assess convergence of MCMC
4. Validate and debug samplers

# Resources

- BDA3:
  - Ch 11
  - Ch 12.2
- BIDA: Ch 6.3.2–6.3.3
- [Metropolis Algorithm demo](#) by Jeffrey Rosenthal

# Metropolis-Hastings

# Metropolis-Hastings Algorithm

- Developed by Nicholas Metropolis, Arianna Rosenbluth, Marshall Rosenbluth, Augusta Teller, and Edward Teller in [1953 paper](#)
- Extended in [1970 by W. K. Hastings](#)
- Did not reach statistical mainstream until early 1990s ([Tierney, 1994](#))
- Very general-purpose sampler, is the basis for most MCMC used today

# Metropolis

## Metropolis Algorithm

### Input:

- Target density (or mass)  $\pi(\boldsymbol{\theta}) \propto g(\boldsymbol{\theta})$
- Symmetric proposal distribution  $Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}')$
- Initial value  $\boldsymbol{\theta}^{(0)}$

For  $s$  in  $1, \dots, S$ , repeat:

1. Draw candidate  $\boldsymbol{\theta}^* \sim Q(\cdot \mid \boldsymbol{\theta}^{(s-1)})$
2. Compute  $\alpha := g(\boldsymbol{\theta}^*) / g(\boldsymbol{\theta}^{(s-1)})$
3. With probability  $\min(1, \alpha)$ , set  $\boldsymbol{\theta}^{(s)} \leftarrow \boldsymbol{\theta}^*$ ; otherwise, set  $\boldsymbol{\theta}^{(s)} \leftarrow \boldsymbol{\theta}^{(s-1)}$

### Output:

Chain  $(\boldsymbol{\theta}^{(1)}, \dots, \boldsymbol{\theta}^{(S)})$

Computation is typically done on the logarithmic scale (with  $\log(g(\boldsymbol{\theta}))$ )

# Metropolis-Hastings

## Metropolis-Hastings Algorithm

### Input:

- Target density (or mass)  $\pi(\boldsymbol{\theta}) \propto g(\boldsymbol{\theta})$
- Generic proposal distribution  $Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}')$  with density (or mass)  $q(\boldsymbol{\theta} \mid \boldsymbol{\theta}')$
- Initial value  $\boldsymbol{\theta}^{(0)}$

For  $s$  in  $1, \dots, S$ , repeat:

1. Draw candidate  $\boldsymbol{\theta}^* \sim Q(\cdot \mid \boldsymbol{\theta}^{(s-1)})$
2. Compute  $\alpha := \frac{g(\boldsymbol{\theta}^*)/g(\boldsymbol{\theta}^{(s-1)})}{q(\boldsymbol{\theta}^* \mid \boldsymbol{\theta}^{(s-1)})/q(\boldsymbol{\theta}^{(s-1)} \mid \boldsymbol{\theta}^*)} = \frac{g(\boldsymbol{\theta}^*) \times q(\boldsymbol{\theta}^{(s-1)} \mid \boldsymbol{\theta}^*)}{g(\boldsymbol{\theta}^{(s-1)}) \times q(\boldsymbol{\theta}^* \mid \boldsymbol{\theta}^{(s-1)})}$
3. With probability  $\min(1, \alpha)$ , set  $\boldsymbol{\theta}^{(s)} \leftarrow \boldsymbol{\theta}^*$ ; otherwise, set  $\boldsymbol{\theta}^{(s)} \leftarrow \boldsymbol{\theta}^{(s-1)}$

### Output:

Chain  $(\boldsymbol{\theta}^{(1)}, \dots, \boldsymbol{\theta}^{(S)})$

# Metropolis-Hastings

Notice:

- The  $\alpha$  ratio eliminates the need to know the normalizing constant for  $g(\cdot)$
- The same is true of  $q(\cdot \mid \cdot)$
- Still need to be able to draw from proposal distribution  $Q(\cdot \mid \cdot)$
- We won't show reversibility or ergodicity, but we can see them in the construction
- Intuition for validity of algorithm outlined in BDA3, more details in BIDA



# Metropolis-Hastings

Flavors of proposal distributions

**Random walk:**  $Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}')$  is usually centered on  $\boldsymbol{\theta}'$

**Informed proposals:**  $Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}')$  is biased to propose “uphill” (typically assisted by gradient of  $g(\boldsymbol{\theta})$ )

**Independence:** Proposal is static  $Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}') = Q(\boldsymbol{\theta})$  and does not depend on the previous state

In the independence sampler:

- Acceptance ratio becomes  $\alpha = \frac{g(\boldsymbol{\theta}^*) \times q(\boldsymbol{\theta}^{(s-1)})}{g(\boldsymbol{\theta}^{(s-1)}) \times q(\boldsymbol{\theta}^*)}$
- $Q(\boldsymbol{\theta})$  should be a reasonable approximation to the target
- Works for one-dimensional targets, does not scale to high dimensions well

# Metropolis-Hastings

## Example

Monthly supercomputer failure counts:  $y \leftarrow c(3, 4, 3, 1, 1, 4, 1, 4, 2, 1, 1, 3, 1, 1, 4)$ .

Assume  $y_i \mid \lambda \stackrel{\text{iid}}{\sim} \text{Pois}(\lambda)$  and  $\lambda \sim \text{Normal}(1, 4^2) 1\{0 < \lambda < 10\}$ .

Sample from the posterior distribution of  $\lambda$ .

$$p(y_1, \dots, y_n, \lambda) = \left[ \prod_{i=1}^n \frac{1}{y_i!} e^{-\lambda} \lambda^{y_i} \right] \frac{(2\pi 4^2)^{-\frac{1}{2}} \exp \left[ -\frac{1}{2 \cdot 4^2} (\lambda - 1)^2 \right] 1(0 < \lambda < 10)}{\int_0^{10} \text{above } d\lambda}$$

$$p(\lambda | y) \propto e^{-n\lambda} \lambda^{\sum y_i} e^{-\frac{1}{2 \cdot 4^2} (\lambda - 1)^2} 1(0 < \lambda < 10)$$

# Burn-in (warm-up)

Discard pre-convergence samples.

- How to determine when convergence occurs? (More to come)

## Example

With the supercomputer failures model:

- Rerun the chain from various starting places
- Visually estimate with trace plots when convergence occurs in each chain

# Autocorrelation

Because we are simulating from a Markov chain, the samples are **not** independent.

## Definition

The **autocovariance** of a stationary Markov chain at lag  $\ell$  is

$$\text{Cov}(X_t, X_{t-\ell}) = \mathbb{E}[(X_t - \mu)(X_{t-\ell} - \mu)]$$

where  $\mu = \mathbb{E}(X_t)$ .

The **autocorrelation** at lag  $\ell$  is then

$$\rho_\ell := \text{Cor}(X_t, X_{t-\ell}) = \frac{\text{Cov}(X_t, X_{t-\ell})}{\text{Var}(X_t)} \in [-1, 1].$$

# Autocorrelation

One consequence of dependence in an MCMC chain is that the *effective sample size* (ESS) is less than  $S$ .

Suppose we wish to estimate  $\mathbb{E}[h(\boldsymbol{\theta})] \approx \overline{h(\boldsymbol{\theta})} = \sum_{s=1}^S h(\boldsymbol{\theta}^{(s)})/S$ .

If the samples were independent, we would have

$$\text{Var}(\overline{h(\boldsymbol{\theta})}) = \frac{\text{Var}(h(\boldsymbol{\theta}))}{S} \quad \text{and} \quad \lim_{S \rightarrow \infty} S \text{Var}(\overline{h(\boldsymbol{\theta})}) = \text{Var}(h(\boldsymbol{\theta}))$$

With autocorrelated samples, this becomes

$$\lim_{S \rightarrow \infty} S \text{Var}(\overline{h(\boldsymbol{\theta})}) = \left(1 + 2 \sum_{\ell=1}^{\infty} \rho_{\ell}\right) \text{Var}(h(\boldsymbol{\theta}))$$

so

$$\lim_{S \rightarrow \infty} \frac{S}{1 + 2 \sum_{\ell=1}^{\infty} \rho_{\ell}} \text{Var}(\overline{h(\boldsymbol{\theta})}) = \text{Var}(h(\boldsymbol{\theta}))$$

# Effective sample size

## Definition

The **effective sample size** of a simulated Markov chain (at stationarity) is

$$n_{\text{eff}} = \frac{S}{1 + 2 \sum_{\ell=1}^{\infty} \rho_{\ell}} .$$

See Ch 11.5 of BDA3 for details of finite-sample approximation.

In practice, we can examine ACF (autocorrelation function) plots and approximate  $n_{\text{eff}}$ .

In R: `acf(chain)`

```
posterior::eff_mean(chain),posterior::eff_bulk(chain),  
posterior::eff_tail(chain),coda::effectiveSize(chain)
```

# Autocorrelation

## Example

With the supercomputer failures model:

- Rerun the chain with various choices of proposal distribution  $Q$ .
- Create ACF plots and estimate  $n_{\text{eff}}$ .

# Thinning

It is common to **thin** MCMC chains by subsampling the chain (usually in a systematic way).

## Example

With the supercomputer failures model:

- Thin an MCMC chain to every 2nd iteration and check ACF plots and estimate of  $n_{\text{eff}}$ .
- Repeat by thinning to every 5th, 10th, 50th iteration.

## Note

Thinning MCMC chains does not improve estimation. It only gives you a higher effective sample size for a fixed number of saved samples.



# Tuning

**Tuning** refers to adjusting the algorithm (in M-H,  $Q$ ) to improve performance of the chain by

- accelerating convergence, or
- decreasing autocorrelation.

# Tuning

## Tuning Metropolis-Hastings

With the random-walk M-H sampler, what tuning parameters are available to us?

*Variance of proposal distribution*

How about the independence M-H sampler?

*Proposal distribution itself*

## Tuning random-walk M-H

Optimal acceptance rate

with univariate  $\theta$ : ~40-50%.

with multivariate  $\theta$ : ~23%.

(See [Gelman, Gilks, and Roberts, 1997](#))

# Adaptive Metropolis-Hastings

After an early run of the chain, we can estimate

- $\mathbb{E}(\boldsymbol{\theta}) := \boldsymbol{\mu}$  with  $\hat{\boldsymbol{\mu}}$  (sample mean) and
- $\text{Cov}(\boldsymbol{\theta}_j, \boldsymbol{\theta}_k) := \boldsymbol{\Sigma}_{j,k}$  with  $\hat{\boldsymbol{\Sigma}}_{j,k}$  (sample covariance).

We can use these estimates to improve the proposal distribution  $Q$

- $Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}') = \text{Normal}_D(\boldsymbol{\theta}', c\hat{\boldsymbol{\Sigma}})$  for random walk
- $Q(\boldsymbol{\theta}) = \text{Normal}_D(\hat{\boldsymbol{\mu}}, c\hat{\boldsymbol{\Sigma}})$  (or corresponding multivariate  $t$ ) for independence sampling

Easy to tune for  $c$ , but [Roberts and Rosenthal \(2009\)](#) give suggestions that depend on dimension  $D$ .

# Adaptive Metropolis-Hastings

At some point we need to stop adapting the chain (otherwise it is not a Markov chain).

However, under certain conditions (see [Roberts and Rosenthal, 2009](#)), adaptation may continue and still result in an ergodic chain with the correct stationary distribution.

# Adaptive Metropolis-Hastings

## Example

Implement an adaptive random-walk M-H sampler for  $(\alpha, \beta)$  in the bioassay example (BDA3, Ch 3.7).

Recall the model:  $y_i \stackrel{\text{ind}}{\sim} \text{Binomial}(n_i, \theta_i)$  for  $i = 1, \dots, n$

with  $\log \left( \frac{\theta_i}{1-\theta_i} \right) = \alpha + \beta x_i$ .

# Hybrid Gibbs samplers

# Composition of transition distributions

If  $K_1(x, x')$  and  $K_2(x, x')$  are two Markov transition kernels, we can combine them into a single transition:

$$K(x_0, x_1) = K_1(x_0, x') \times K_2(x', x_1) ,$$

which inherits the properties of both  $K_1$  and  $K_2$ .

# Metropolis within Gibbs

One powerful combination is embedding a Metropolis-Hastings algorithm within a Gibbs sampling step.

- Need only one step of the M-H algorithm(!)

Where might this be useful?



# Metropolis within Gibbs

## Example

Implement a one-at-a-time Metropolis-within-Gibbs sampler for the bioassay example (BDA3, Ch 3.7)

Recall the model:  $y_i \stackrel{\text{ind}}{\sim} \text{Binomial}(n_i, \theta_i)$  for  $i = 1, \dots, n$

with  $\log \left( \frac{\theta_i}{1-\theta_i} \right) = \alpha + \beta x_i$ .

# Convergence diagnostics

# Convergence diagnostics

We cannot definitively declare a chain has converged.

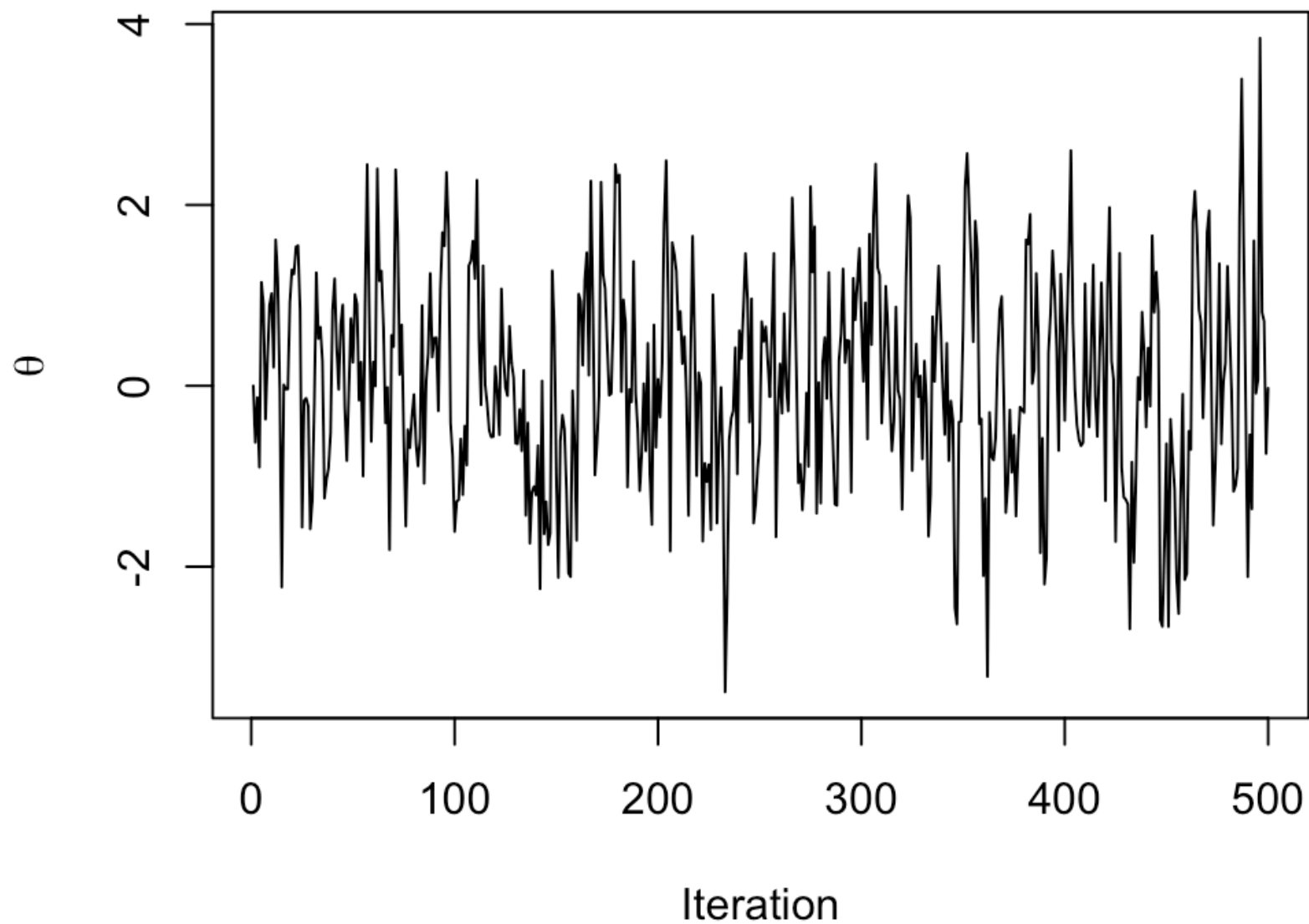
We can only look for the absence of red flags.

Important diagnostic tools:

- Trace plots
- Autocorrelation / effective sample size
- Potential scale reduction factor
- Many others exist

# Trace plots

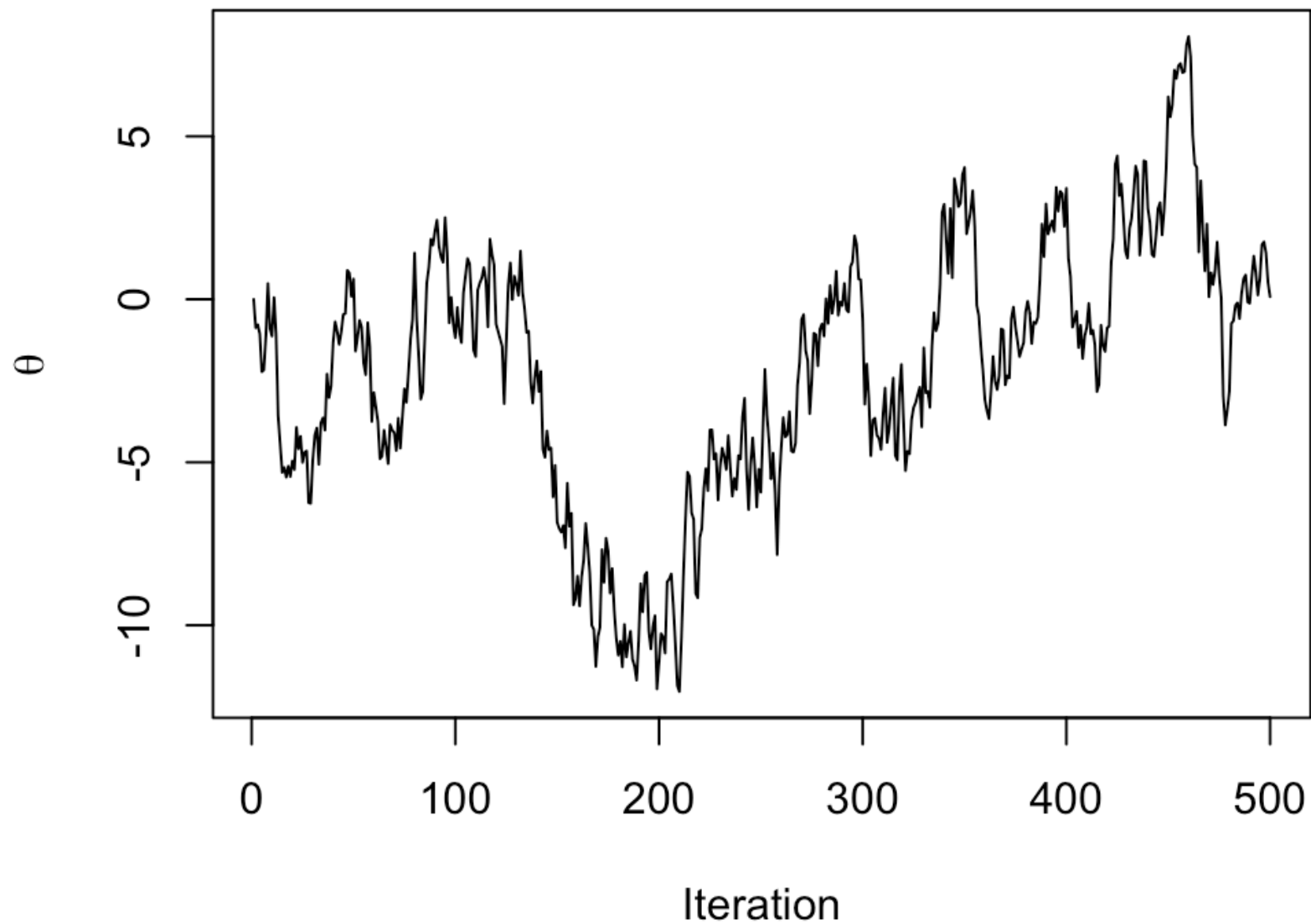
Converged? Guess  $\hat{n}_{\text{eff}}$ :





# Trace plots

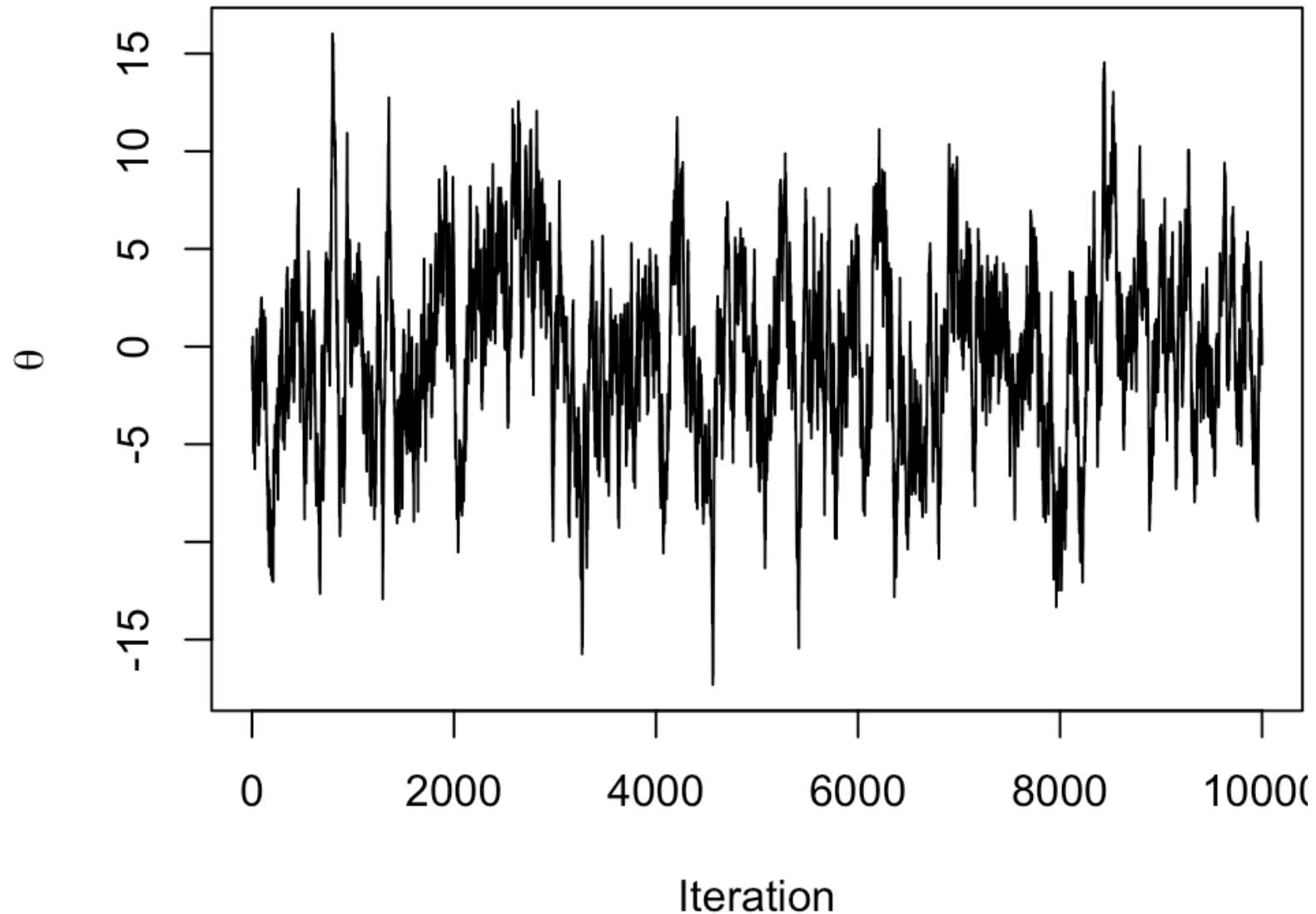
Converged? Guess  $\hat{n}_{\text{eff}}$ :





# Trace plots

Same chain as previous plot. Converged? Guess  $\hat{n}_{\text{eff}}$ :

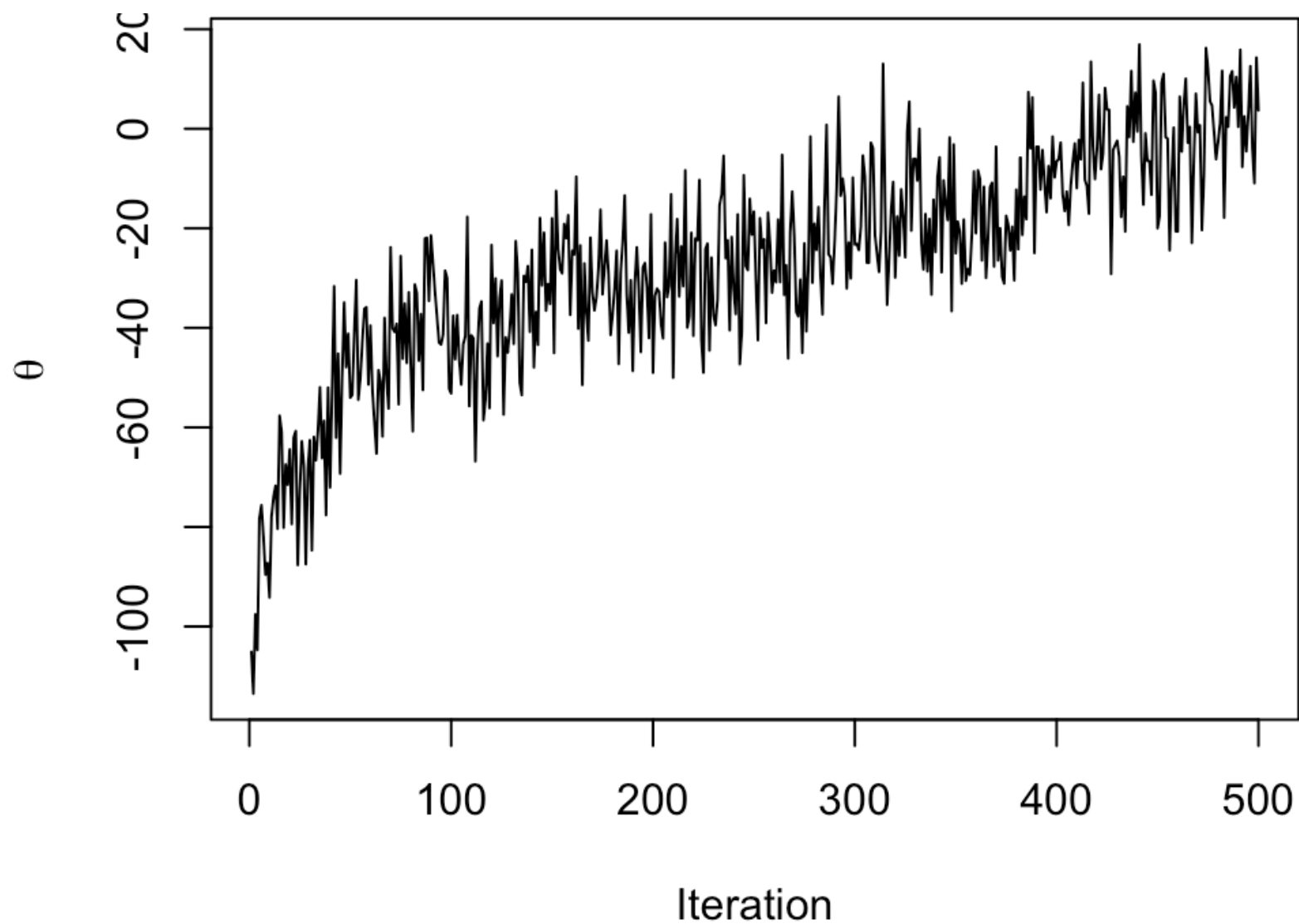






# Trace plots

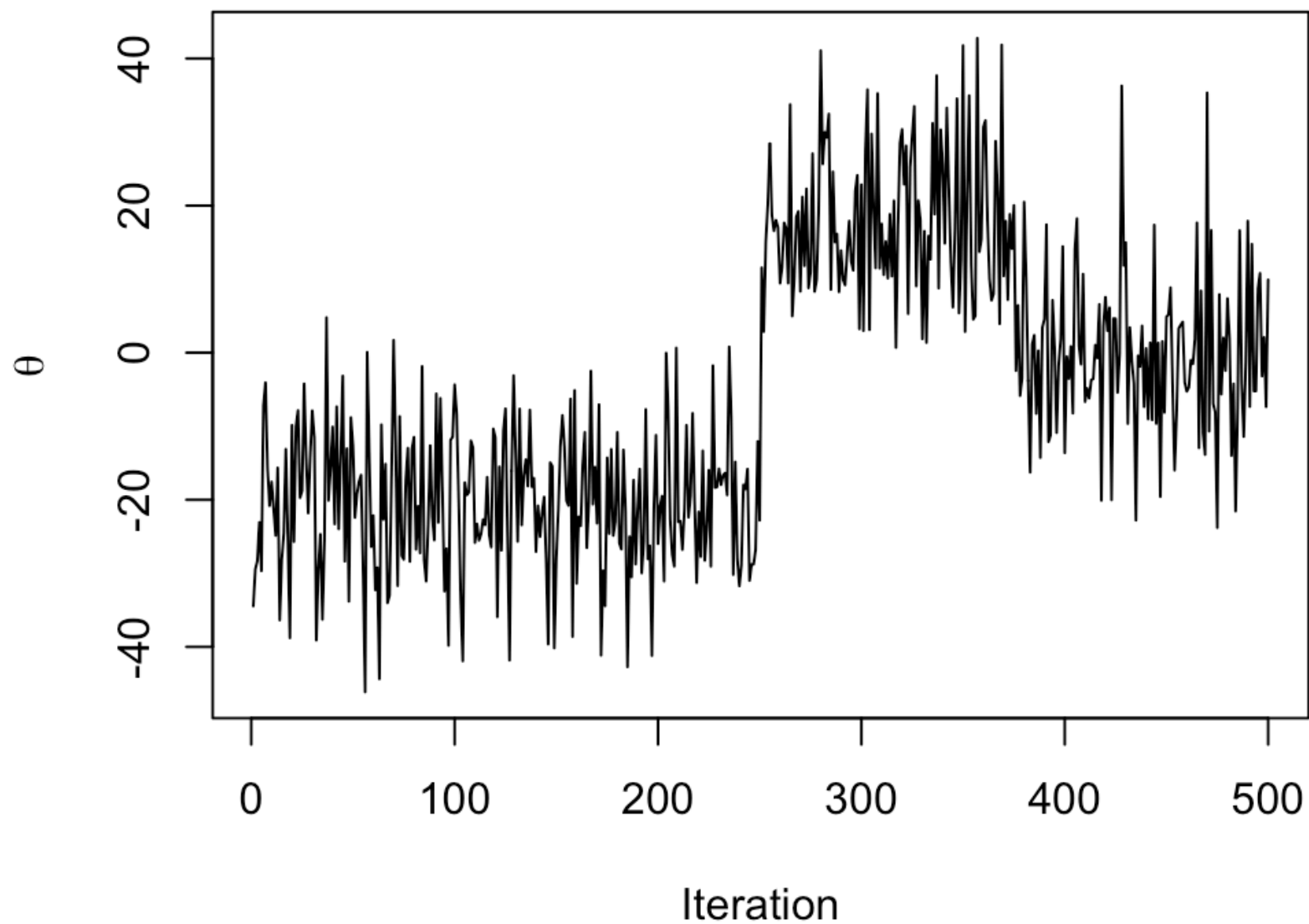
Converged? Guess  $\hat{n}_{\text{eff}}$ :





# Trace plots

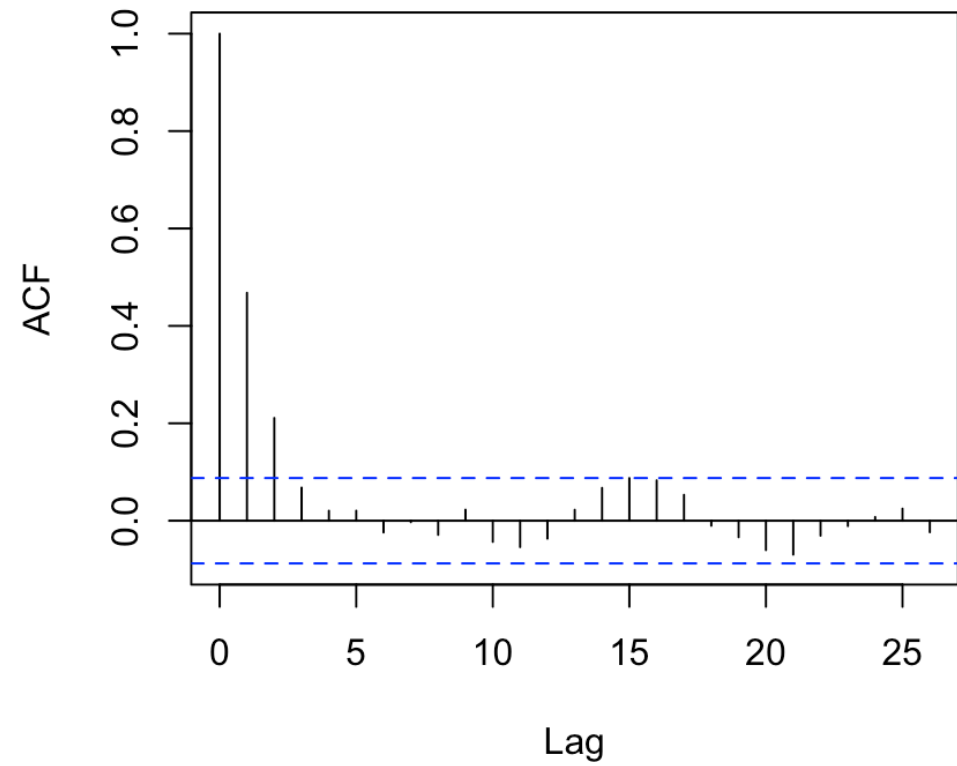
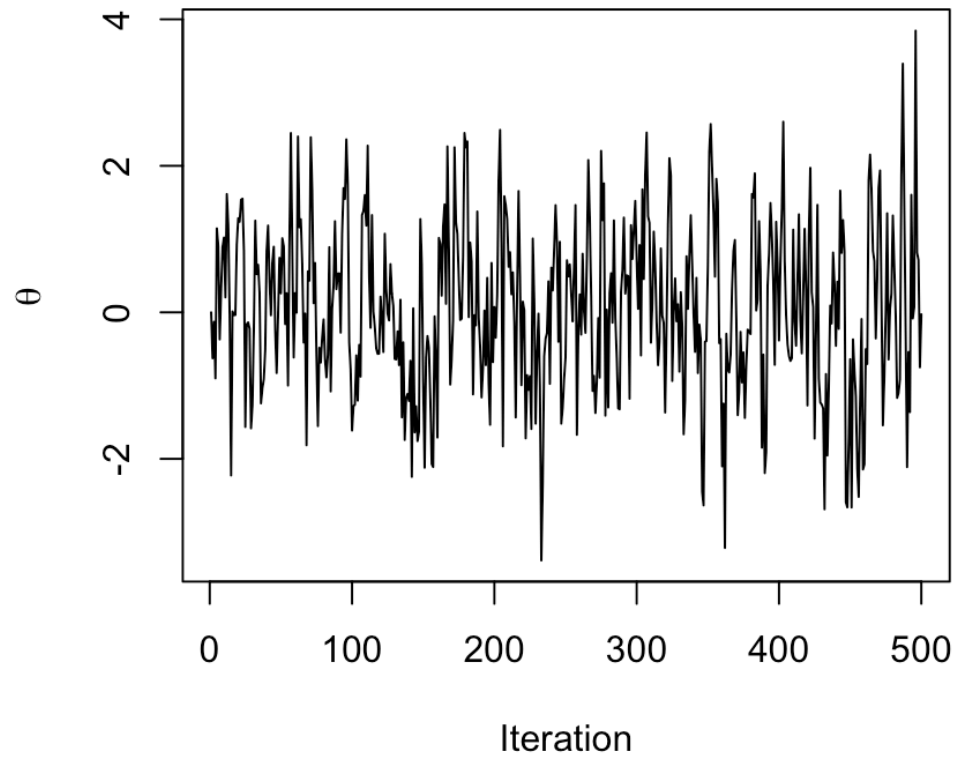
Converged? Guess  $\hat{n}_{\text{eff}}$ :





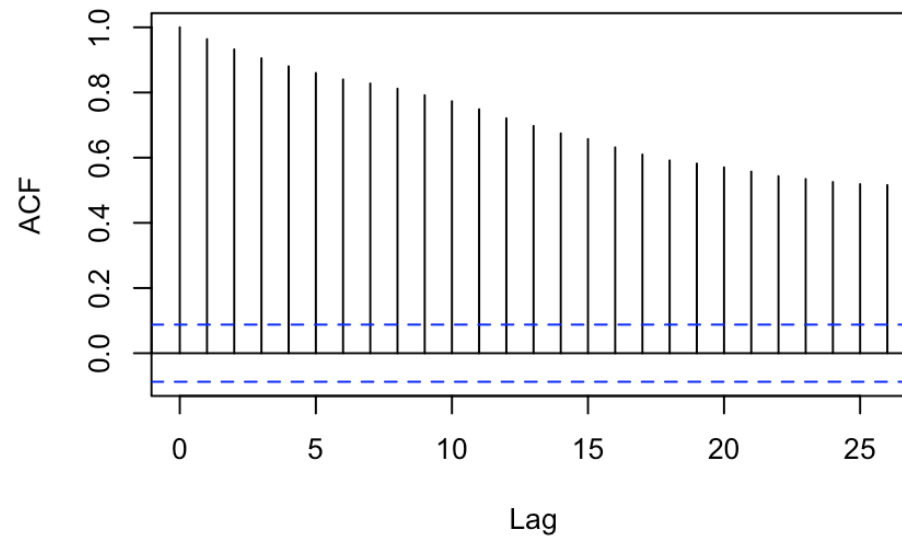
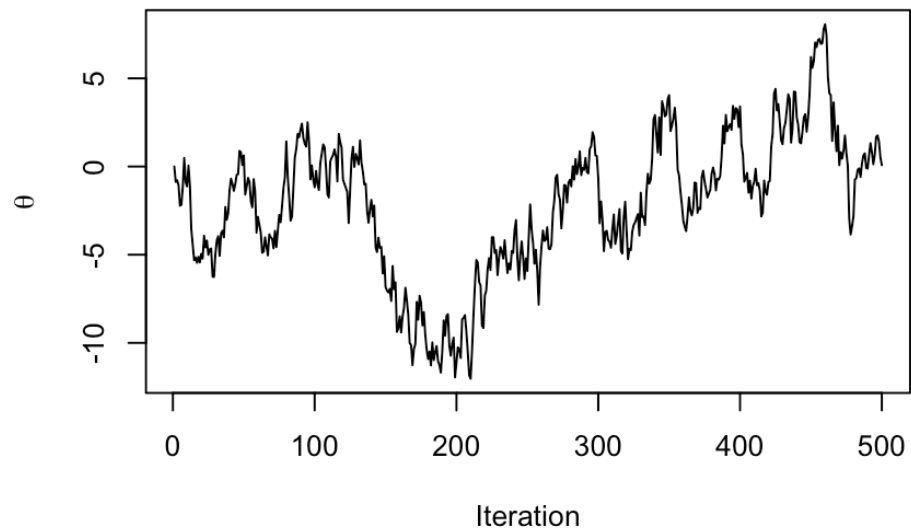
# ACF plots

**n\_eff = 193.9**

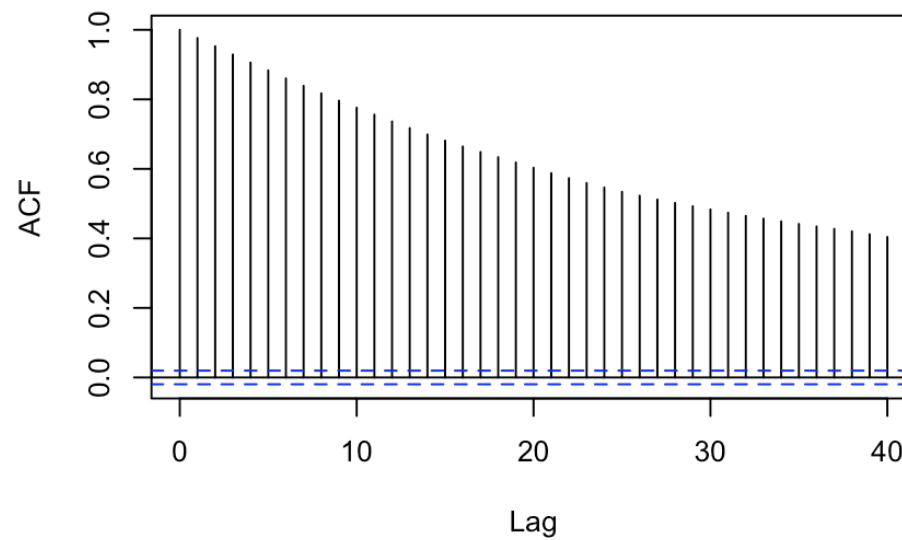
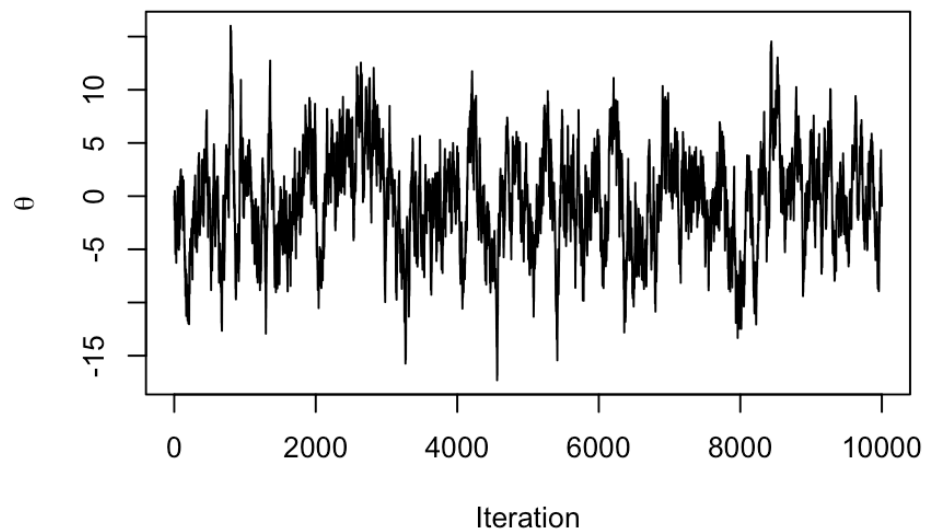


# ACF plots

**n\_eff = 2.7**



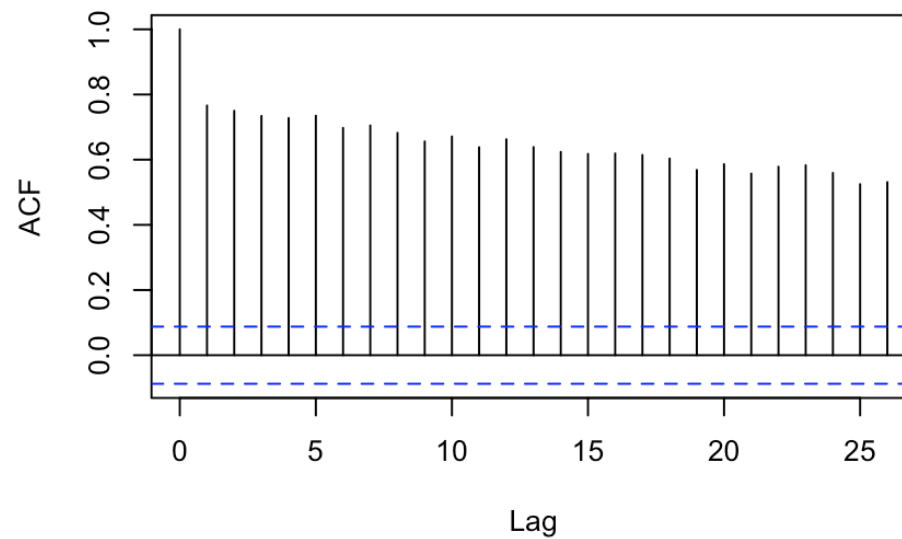
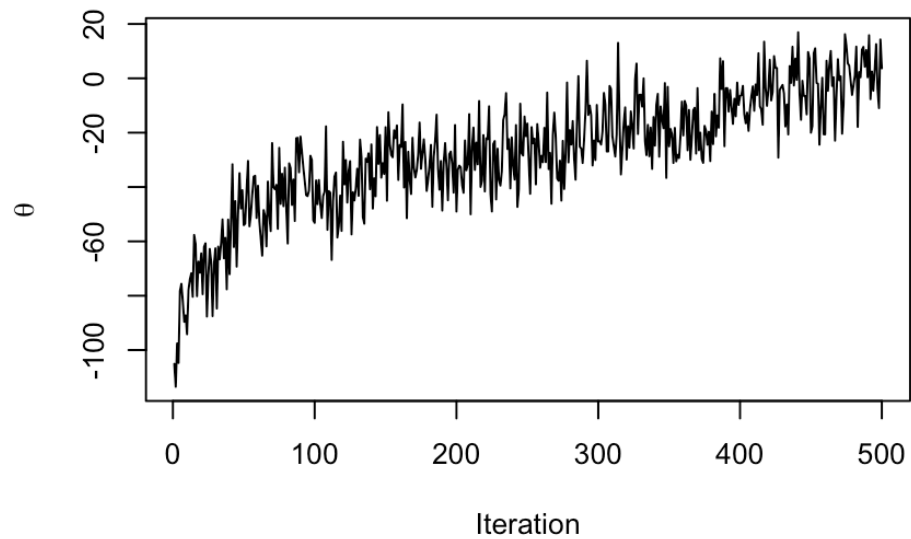
**n\_eff = 112.1**



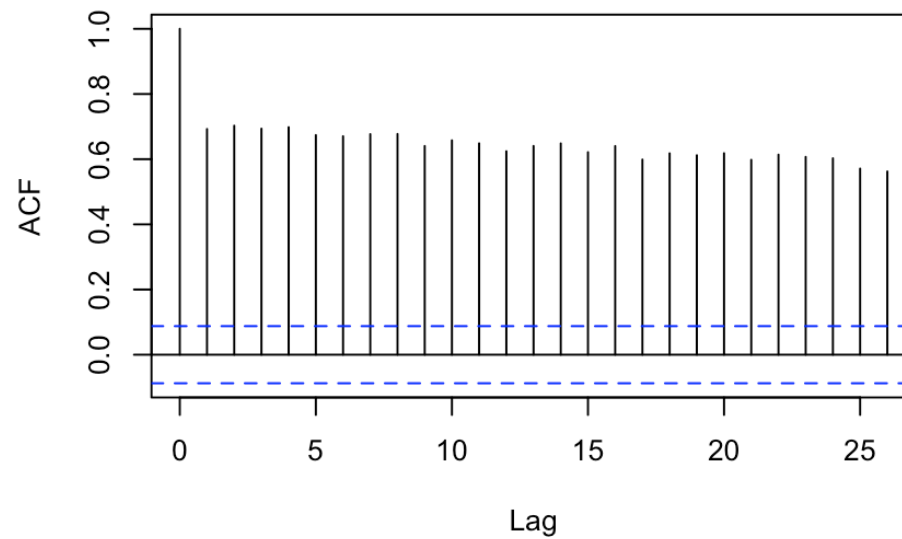
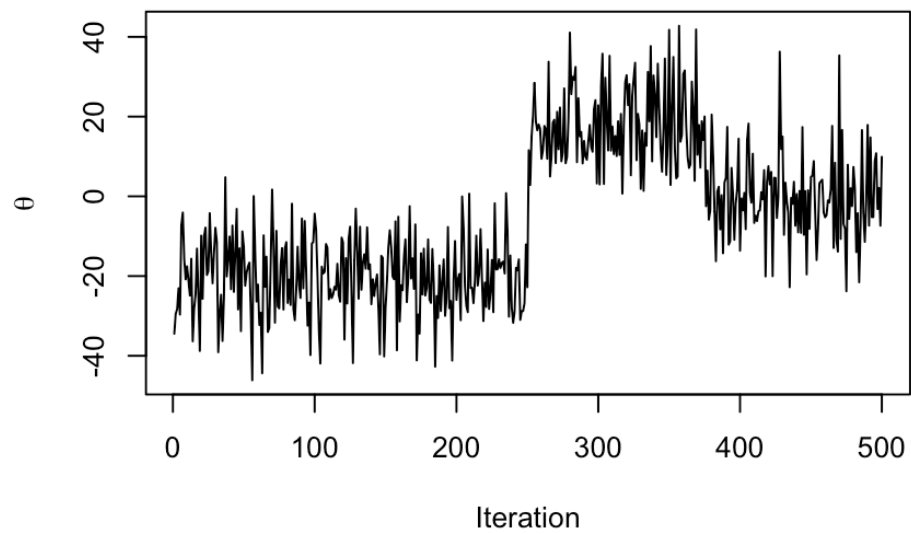


# ACF plots

**n\_eff = 1.6**

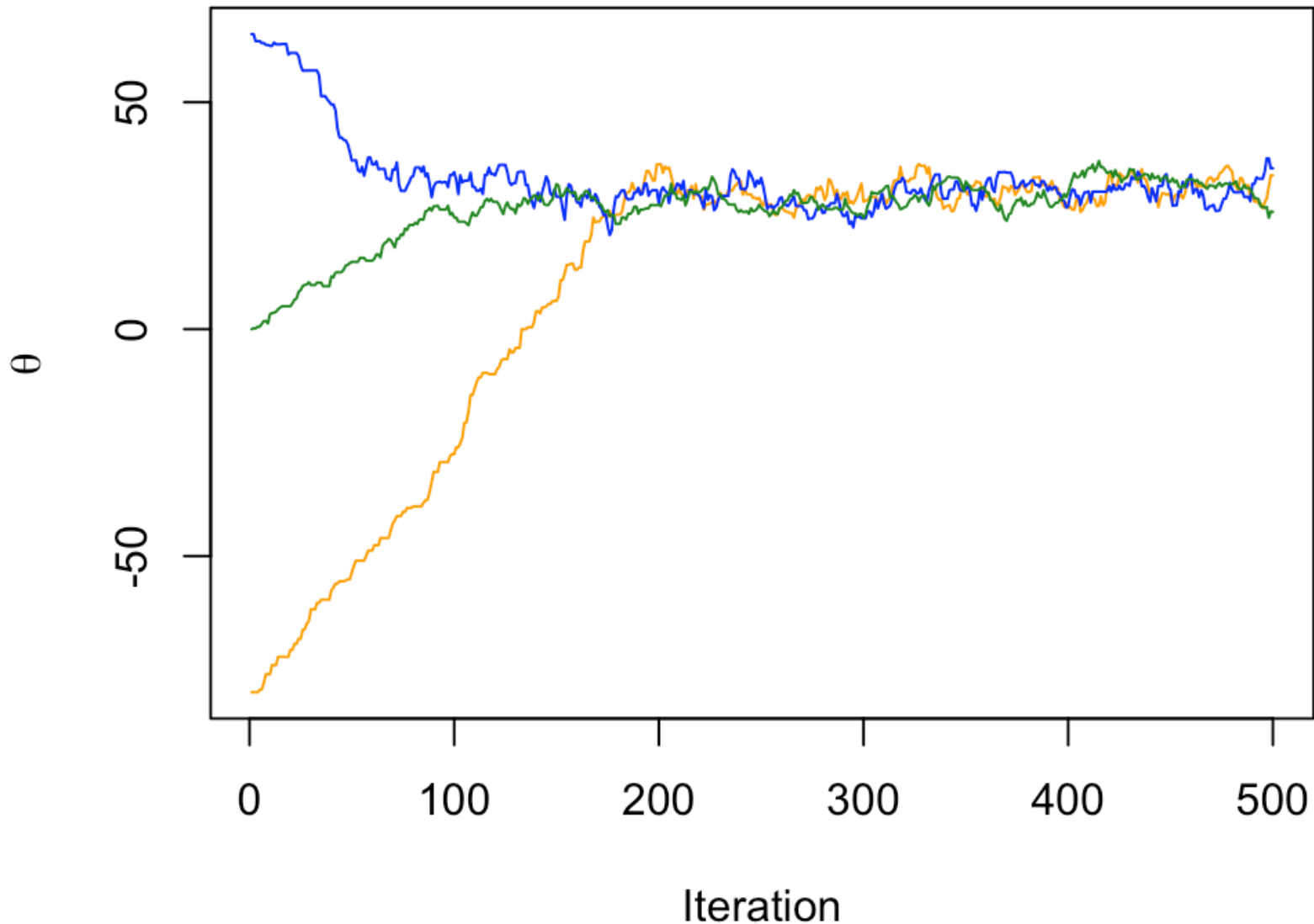


**n\_eff = 1.4**



# Potential scale reduction factor

When does convergence occur?





# Potential scale reduction factor

**Idea:** Compare variability *within* chains to variability *between* chains.

Let

$$B = \frac{1}{M-1} \sum_{m=1}^M (\bar{\theta}_m - \bar{\theta}_{\cdot})^2$$

estimate the variance between  $M$  chains (with respect to a grand mean  $\bar{\theta}_{\cdot}$ ), and

$$W = \frac{1}{M} \sum_{m=1}^M \left[ \frac{1}{S} \sum_{s=1}^S \left( \theta_m^{(s)} - \bar{\theta}_m \right)^2 \right]$$

be an average of within-chain variances.

Note that  $S$  is the chain length *after* discarding burn-in samples.

(Adapted from Ch 11.4 of BDA3)

# Potential scale reduction factor

## Definition

The **potential scale reduction factor** or **R-hat** is

$$\hat{R} = \sqrt{\frac{W + B}{W}},$$

which approaches 1 as  $S \rightarrow \infty$ .

## Notes

- General rule: if  $\hat{R} > 1.10$ , continue the chains.
- BDA3 suggests splitting each chain in half (thus doubling  $M$ ) to also test whether they are still drifting.
- In R: `posterior::rhat(chains_matrix)` or `coda::gelman.diag(mcmc_list_object)`

# Other diagnostic tools

There are many diagnostics to choose from. Two more common diagnostics are listed below.

- [Geweke diagnostic](#): test for equality of means from the first and last half of a Markov chain
- [Raftery diagnostic](#): estimates the number of iterations required to estimate a given quantile of the target distribution

[Cowles and Carlin \(1996\)](#) give a comparative review of convergence diagnostic tools.

# Troubleshooting MCMC



# When your MCMC isn't working...

Remember, there is always a bug in the code. Even when it looks like it's working!

General advice:

- Run multiple chains from dispersed initial values and verify that they all converge to the same place.
- The wisdom in BDA3 Ch 10.7 will save you a *lot* of time and tears.
  - Simulate from the generative model and recover the “truth.”
  - Increasing the data sample size should concentrate the posterior distribution around the truth. If not, also check that the model is identifiable.

# When your MCMC isn't working...

General advice, continued:

- Efficiency of code is not always more important than readability.
- Write modular code (that is, use functions).
  - Solve smaller sub-problems, ideally ones with known solutions, first.
  - Isolate parts to verify that they are each working properly. For example, in a Gibbs sampler, turn off all but one update and make sure each is working in isolation.
  - Instead of pointing update functions to arrays of stored chains, point to a current state object.

